



Инференс нейросетей

Лекция 10





Константин Мосунов

Старший разработчик
Платформы инференса

План лекции

Что такое инференс

Паттерны инференса

Инференс-фреймворки

Инференс-серверы

Платформы инференса



Что такое инференс

Инференс (от англ. *inference* — вывод, заключение) — использование обученной модели для получения предсказаний на новых данных.

Характеристика	Обучение	Инференс
Цель	Поиск оптимальных весов	Получение предсказания
Метрики	Loss, F-score, и т. п.	Latency, throughput, GPU Utilization и т. п.
Потребление ресурсов	Прямой и обратный проход: - веса - градиенты - состояние оптимизатора	Только прямой проход: - веса
Режим работы	Offline	Online / offline
Требования к отказоустойчивости	Низкие / средние	Средние / высокие

Что такое инференс

Паттерны инференса

Инференс-фреймворки

Инференс-серверы

Платформы инференса

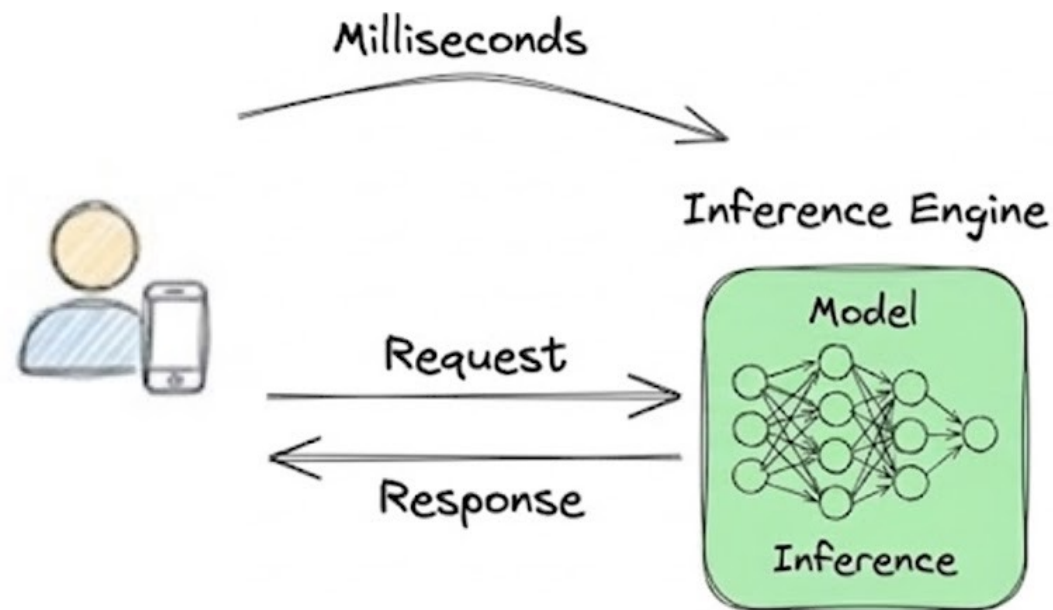


Паттерны инференса

- Синхронный (online);
- Пакетный (batch);
- Поточковый (stream);

Синхронный инференс

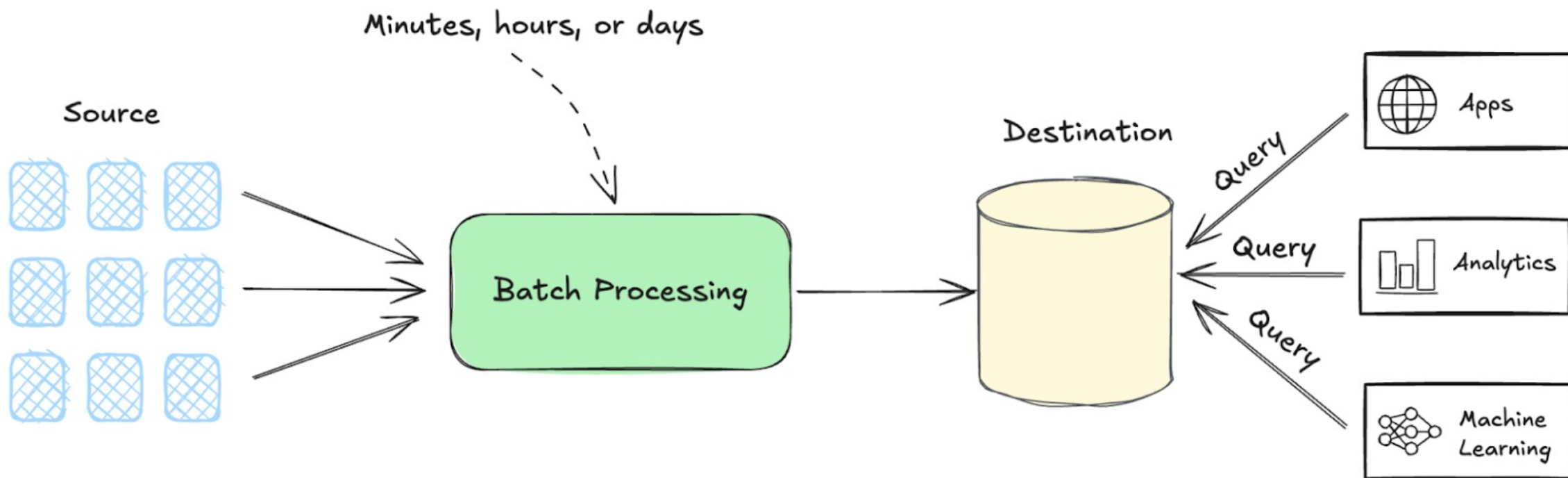
- Запрос – ответ;
- Примеры:
 - поиск;
 - чат-боты;
 - реклама;
- Преимущества:
 - простота реализации;
 - минимальная задержка;
- Недостатки:
 - низкая утилизация GPU;
 - низкая пропускная способность;



Пакетный (batch) инференс

- Накопление данных;
- Регулярный запуск по расписанию;
- Примеры:
 - периодические отчёты, прогнозы;
 - рекомендации (плейлисты, подборки товаров).

Пакетный (batch) инференс



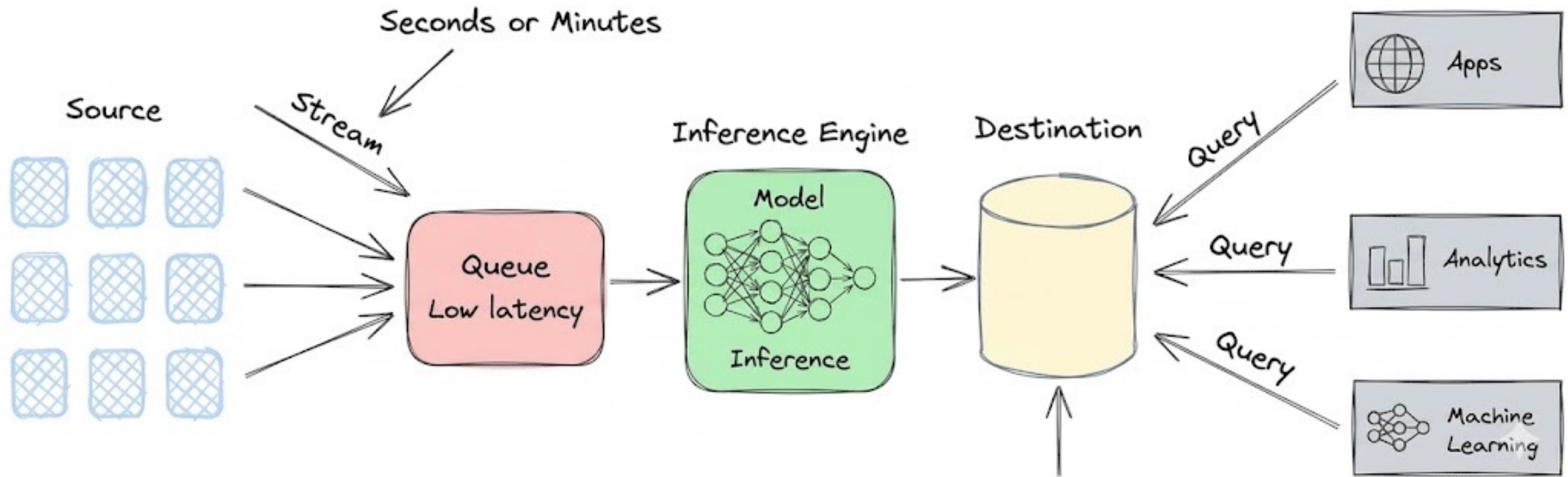
Пакетный (batch) инференс

- Преимущества:
 - максимальная утилизация GPU;
 - максимальная пропускная способность;
- Недостатки:
 - очень высокая задержка;
 - риск потери актуальности данных (рекомендации);

Потоковый (stream) инференс

- Баланс между синхронным и пакетным;
- Поток данных;
- Примеры:
 - модерация контента;
 - антифрод;
 - видеоаналитика (обнаружение пожаров).

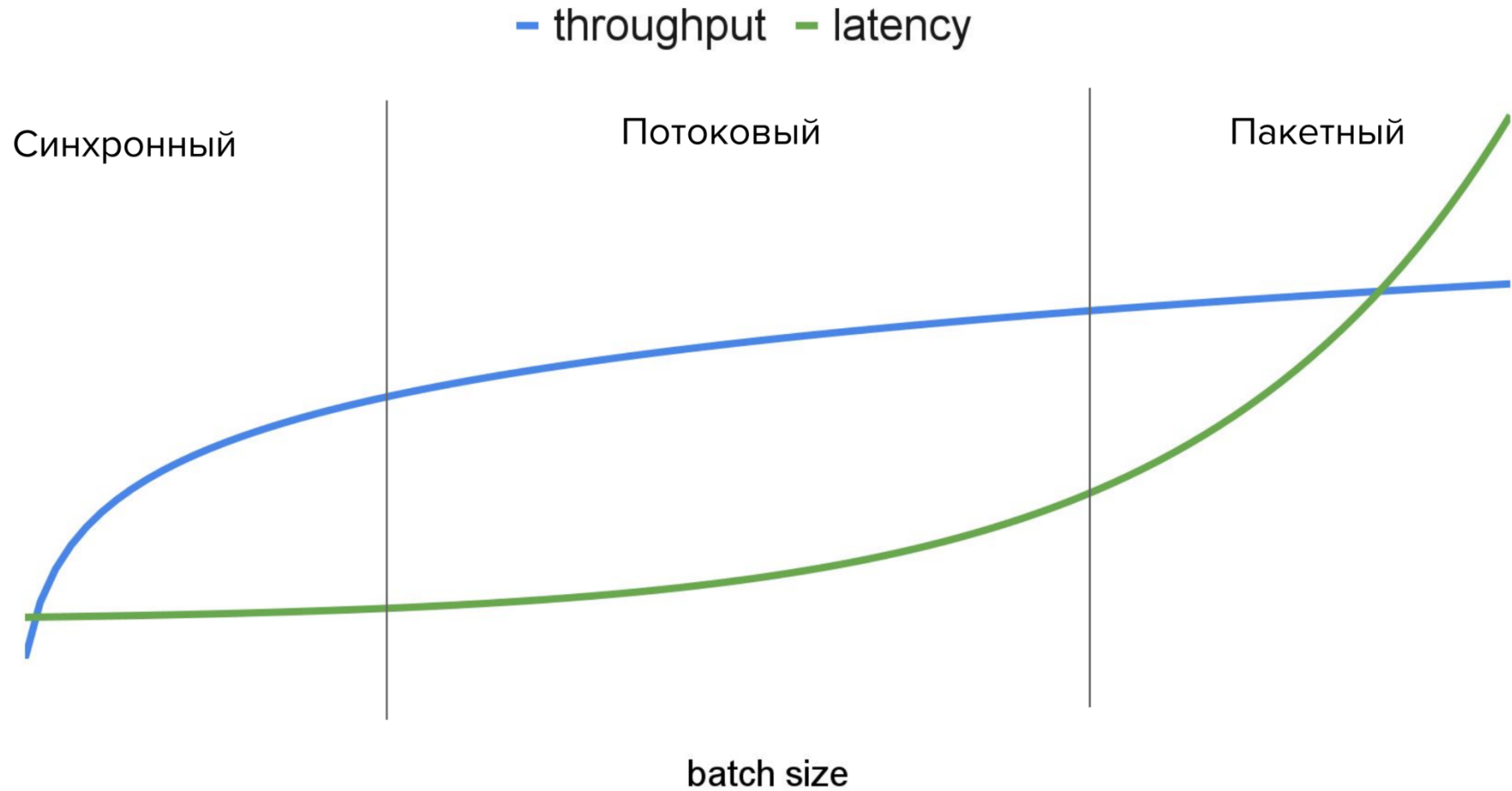
Потоковый (stream) инференс



Потоковый (stream) инференс

- Преимущества:
 - устойчивость к пиковым нагрузкам;
 - хорошая утилизация GPU;
- Недостатки:
 - переменная задержка.

Latency vs Throughput



Что такое инференс

Паттерны инференса

Инференс-фреймворки

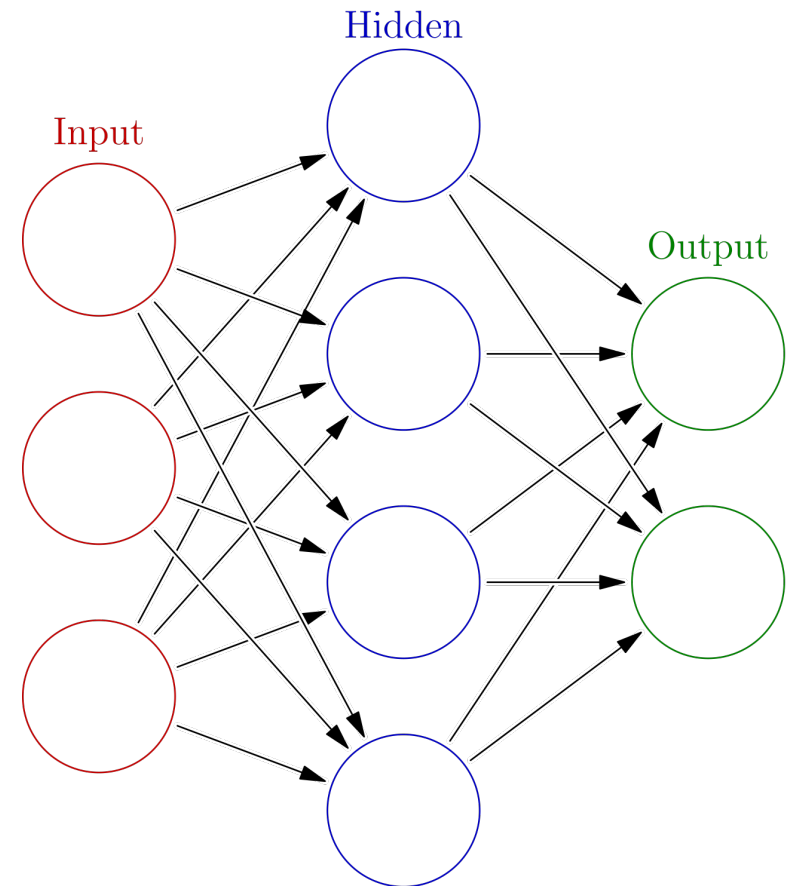
Инференс-серверы

Платформы инференса



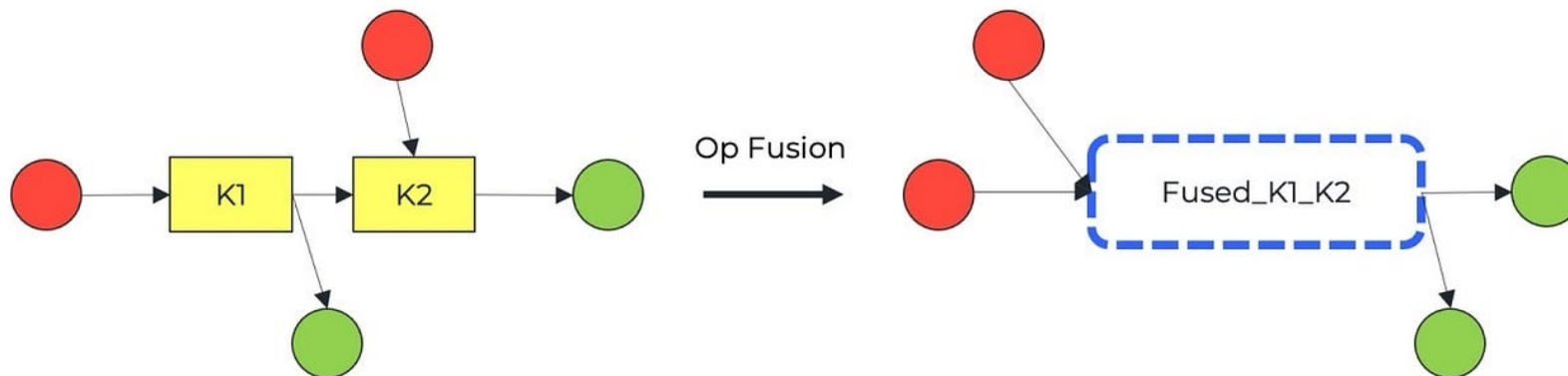
Инференс-фреймворки

- Специализированное ПО для эффективного выполнения на целевом аппаратном обеспечении;
- Формат представления – Intermediate Representation;
- Оптимизации:
 - вычислительного графа;
 - управление памятью (IO bottleneck);
 - квантизация;
- Рантайм:
 - от интерпретации к компиляции;
 - абстрагирует от железа;
- Унификация формата и интерфейса;
- Императивность vs декларативность.

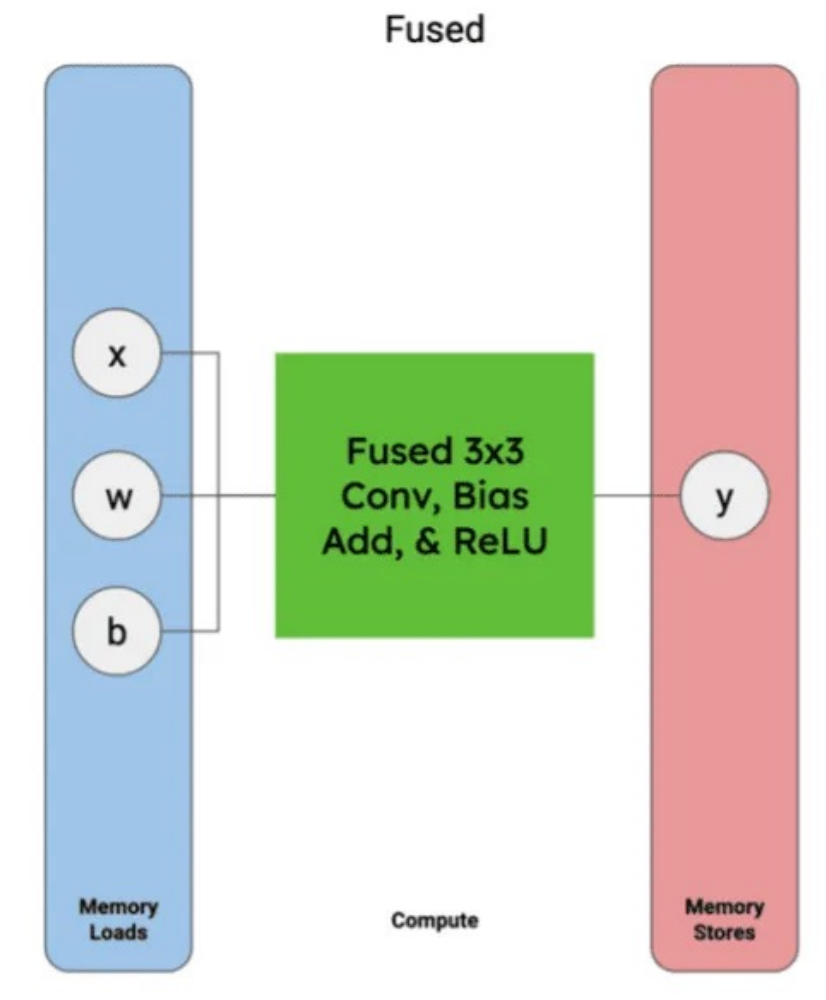
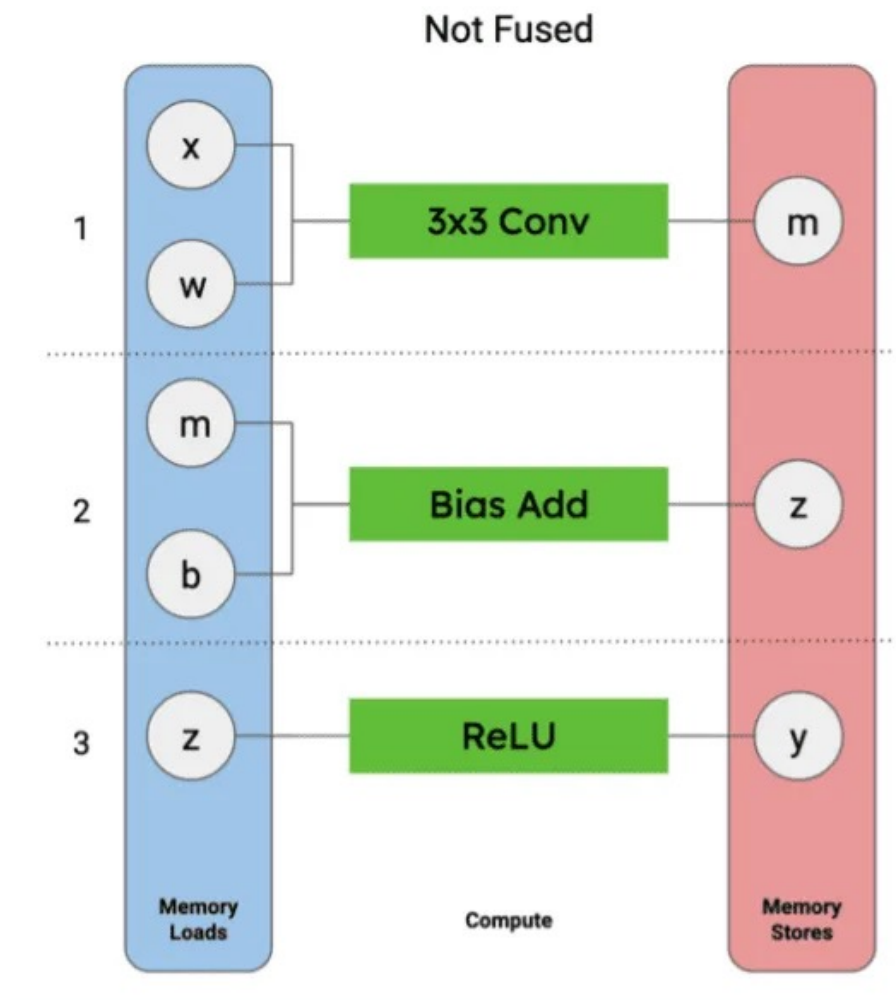


Оптимизация графа вычислений

- Предварительное вычисление констант (constant folding);
- Удаление узлов (node elimination);
- Объединение узлов (node fusion, вертикальное и горизонтальное);
- Оптимизации формата данных (NCHW vs NHWC).



Оптимизация графа вычислений



Пример: Conv + BatchNorm

$$z = W * x + b$$

$$BN(z) = \gamma \cdot \frac{z - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

$$BN(W * x + b) = \gamma \cdot \frac{W * x + b - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

$$\alpha = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}}, \quad \beta' = \beta - \alpha \cdot \mu$$

$$BN(W * x + b) = \alpha \cdot (W * x + b) + \beta - \alpha \cdot \mu = \alpha \cdot W * x + (\alpha \cdot b + \beta')$$

$$W' = \alpha \cdot W, \quad b' = \alpha \cdot b + \beta'$$

$$y = W' * x + b'$$



Инференс-фреймворки

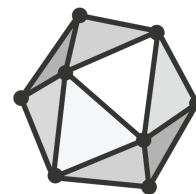
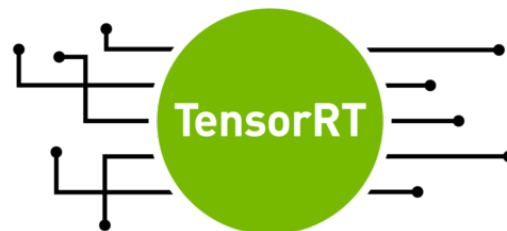
Демо

Выбор фреймворка для инференса

- Быстрая интеграция, от MVP до production сервисов – PyTorch;
- Intel – OpenVINO;
- Nvidia GPU – TensorRT;
- Нужна унификация в условиях разного железа и софта – ONNX;

 PyTorch

 OpenVINO



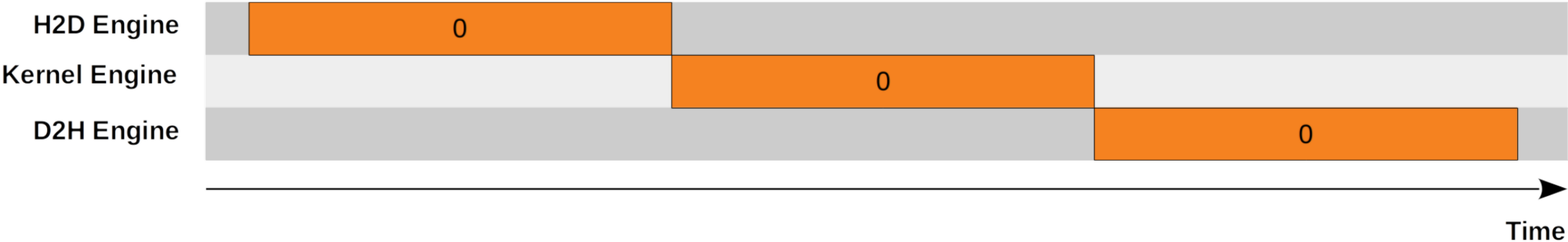
ONNX

Об оптимизациях

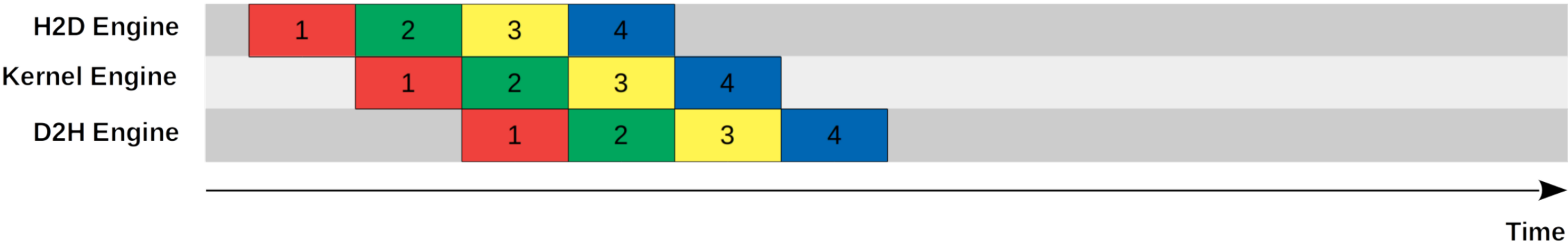
- Локальная vs комплексная оптимизация;
- Поиск узких мест, профилирование;
- Пример – оптимизация ETL пайплайна обработки данных:
 - чтение – обработка – запись;
 - сеть, диск – узкие места. Сжатие и распаковка – меньше IO, больше вычислений;
 - передача данных CPU <-> GPU;
 - смежные инструменты: CUDA Streams и т. п.

Об оптимизациях

Serial Model



Concurrent Model



Что такое инференс

Паттерны инференса

Инференс-фреймворки

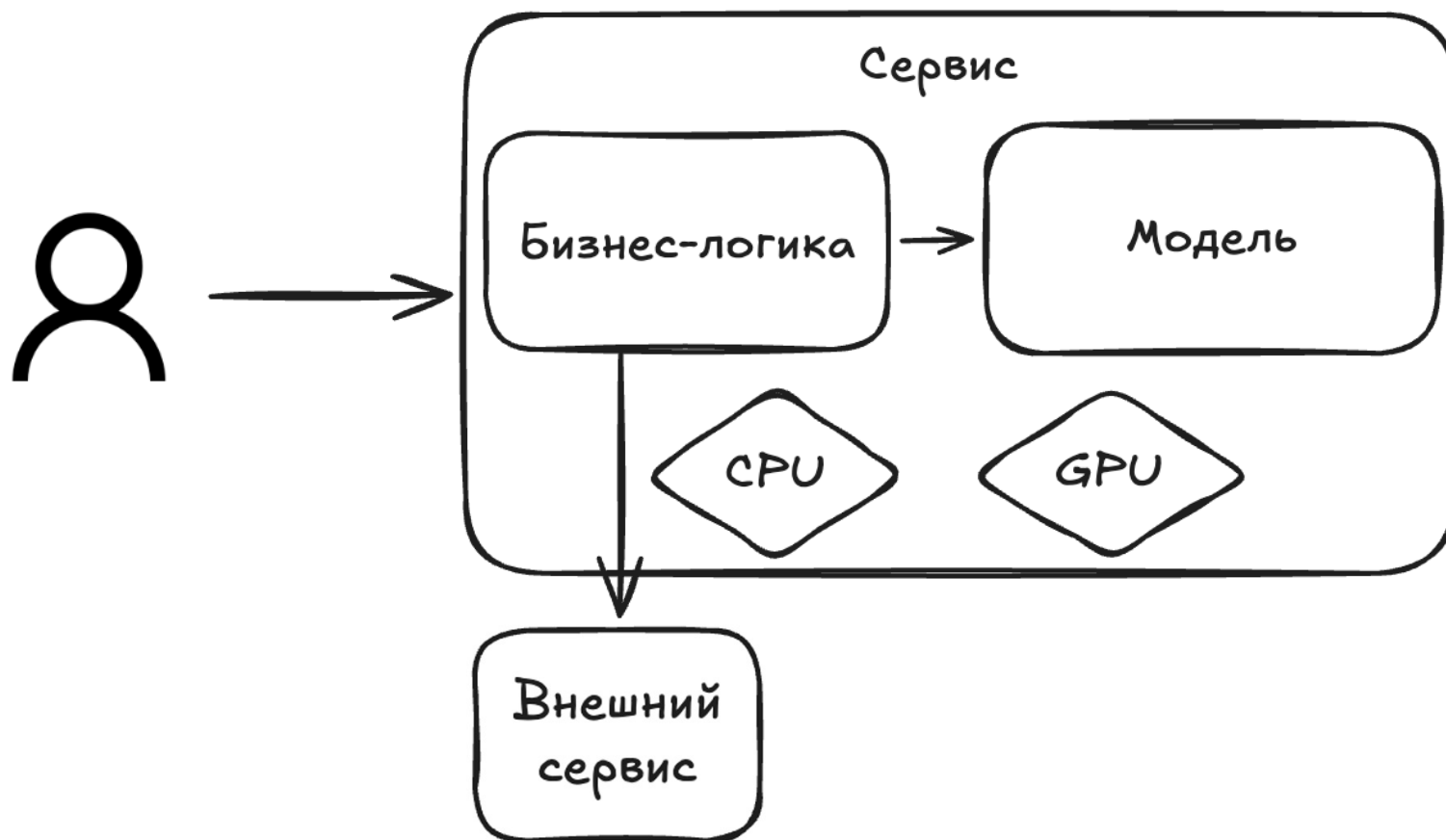
Инференс-серверы

Платформы инференса



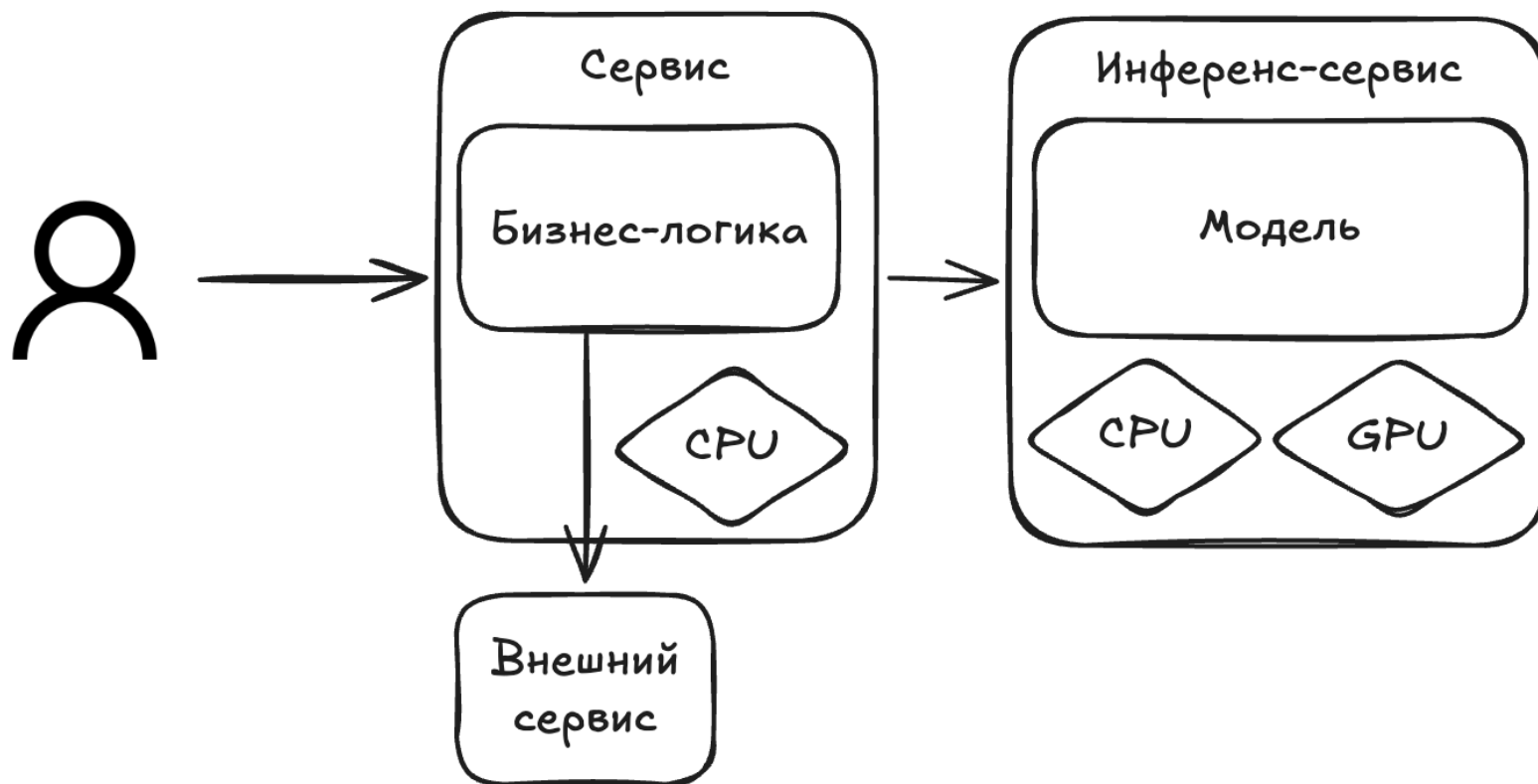
Интеграция модели в сервис

- Как часть уже существующего сервиса:
 - бизнес-логика + модель;
 - ONNX Runtime Python API;
 - нулевые сетевые задержки;
 - сложность масштабирования;
 - сервис на Go, API для C++;



Интеграция модели в сервис

- Как отдельный сервис (Model as a Service):
 - независимое масштабирование и обновление;
 - fault isolation;
 - независимость технологий;
 - сетевой оверхед;



Инференс-серверы

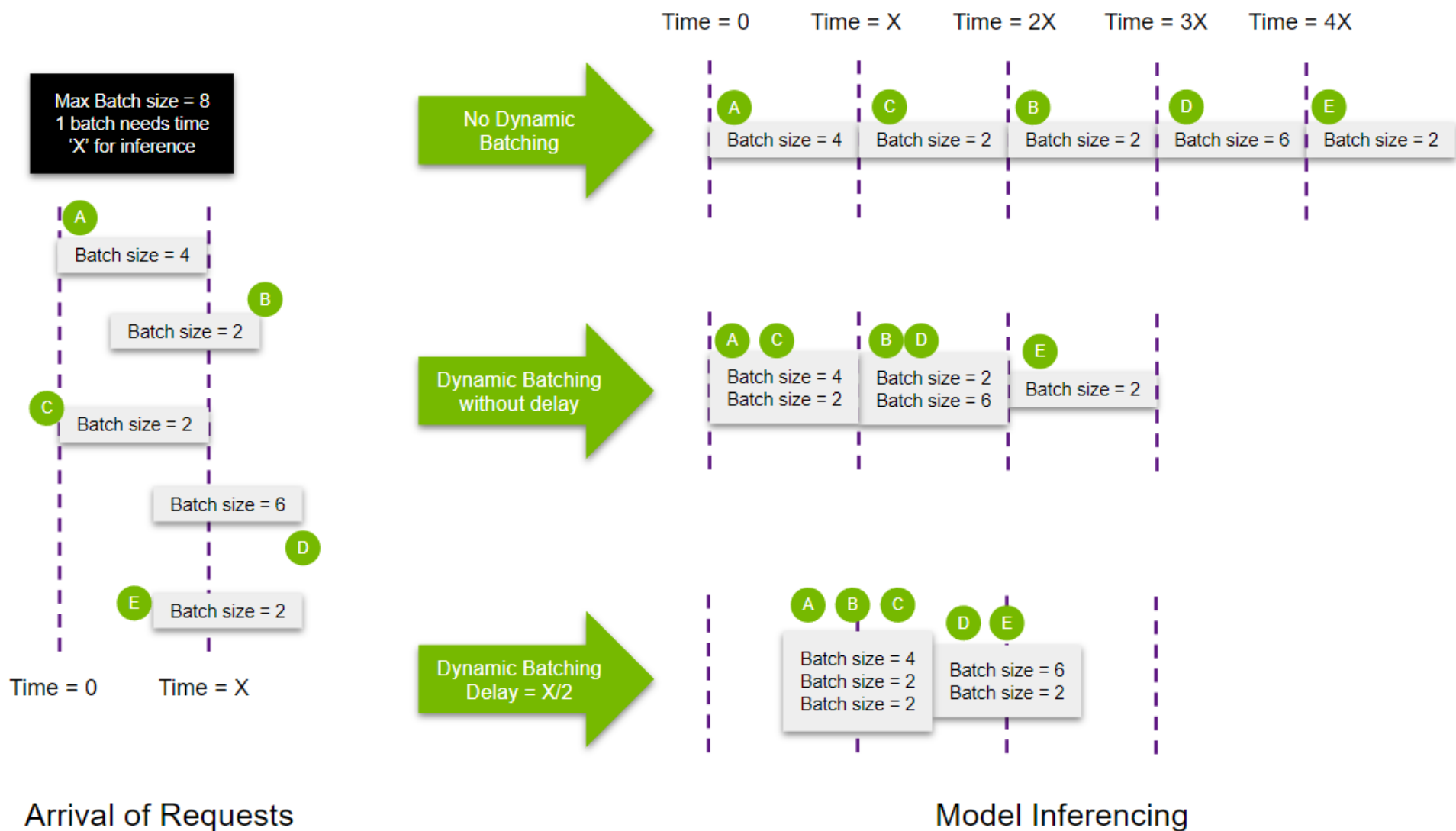
Общие фичи сервиса:

- Сетевой интерфейс (API);
- Диспетчеризация запросов, управление очередями;
- Мониторинг: хелсчеки, метрики;
- Отказоустойчивость;

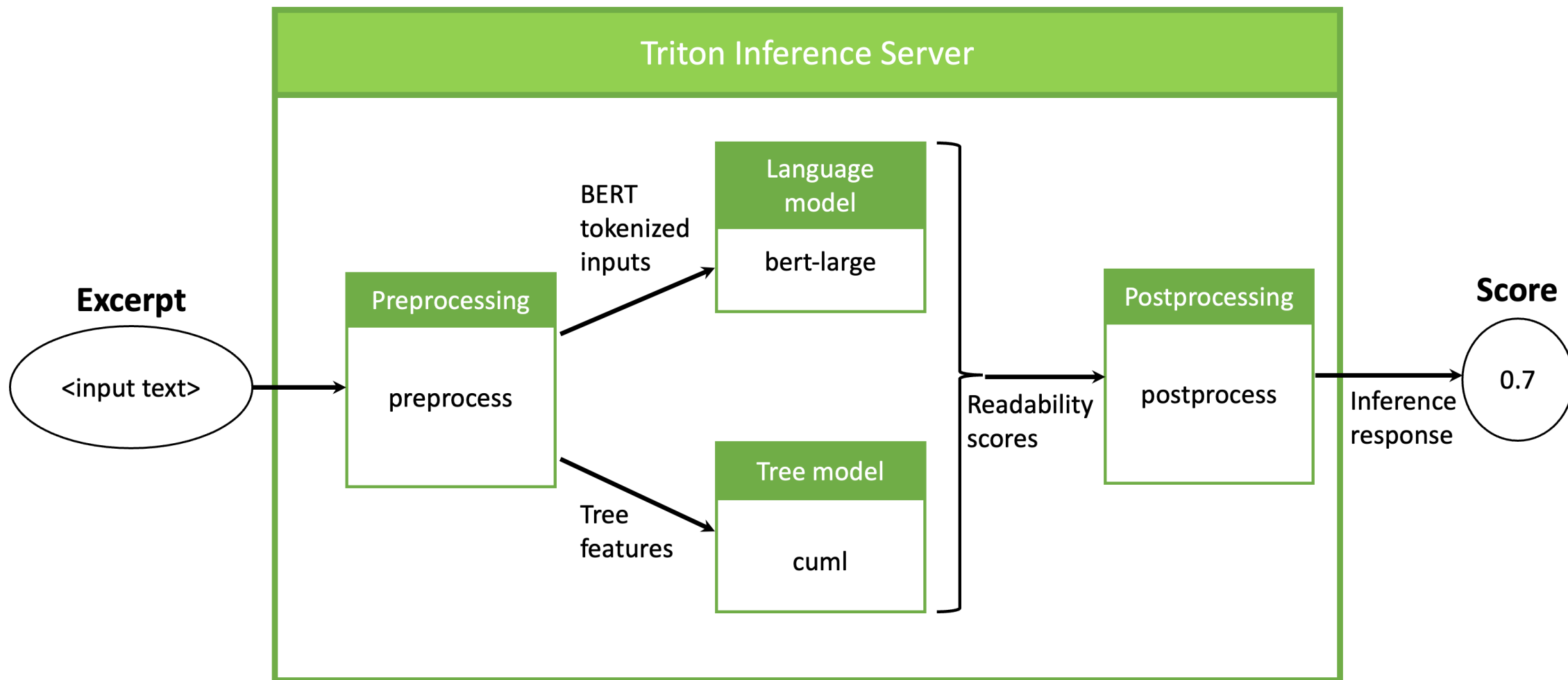
Фичи в контексте ML моделей:

- Управление жизненным циклом моделей;
- Динамический батчинг;
- Параллельное исполнение нескольких экземпляров;
- Ансамбли;

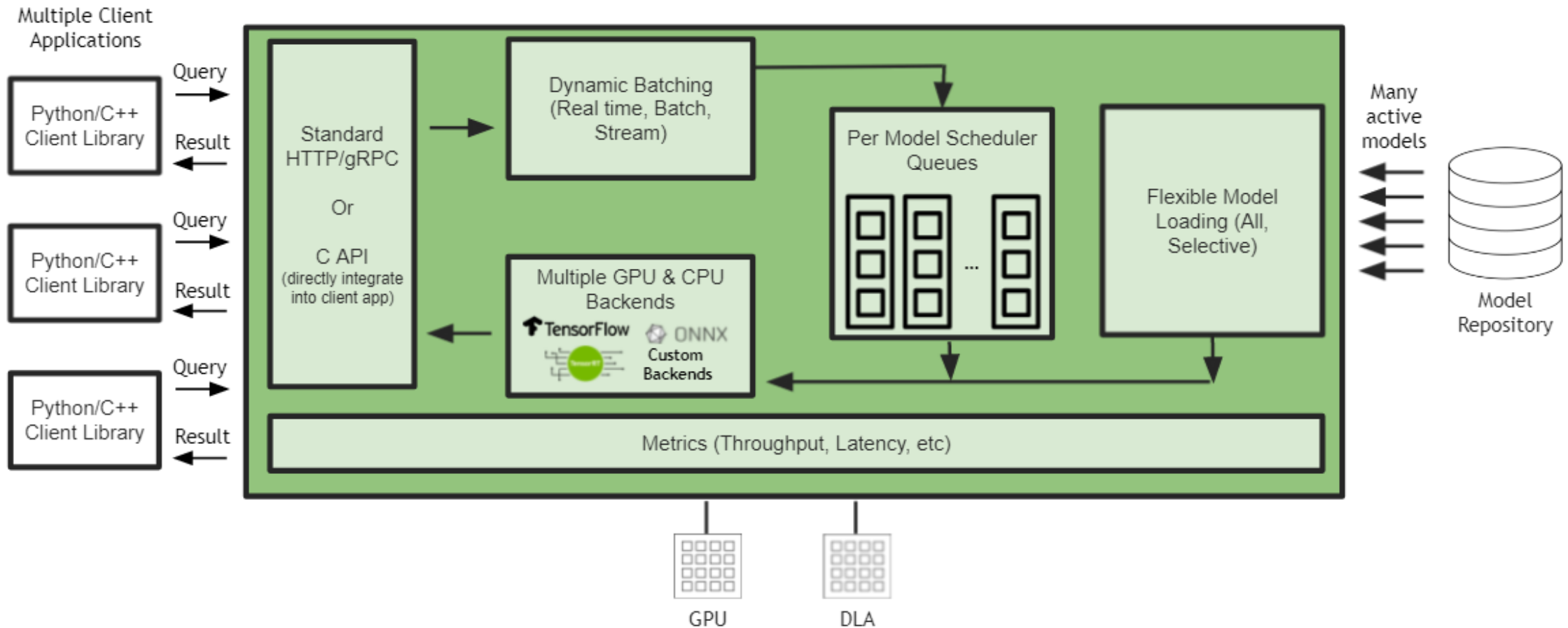
Динамический батчинг



Ансамбли моделей



Triton Inference Server



Triton Inference Server

- Nvidia;
- Много бекендов: Python, PyTorch, ONNX, OpenVINO, TensorRT и т.п.
- API: gRPC / REST, Open Inference Protocol;
- Динамический батчинг;
- Кэширование ответов;
- Метрики для Prometheus;
- Triton SDK: Model Analyzer;



Triton Inference Server Демо

Что такое инференс

Паттерны инференса

Инференс-фреймворки

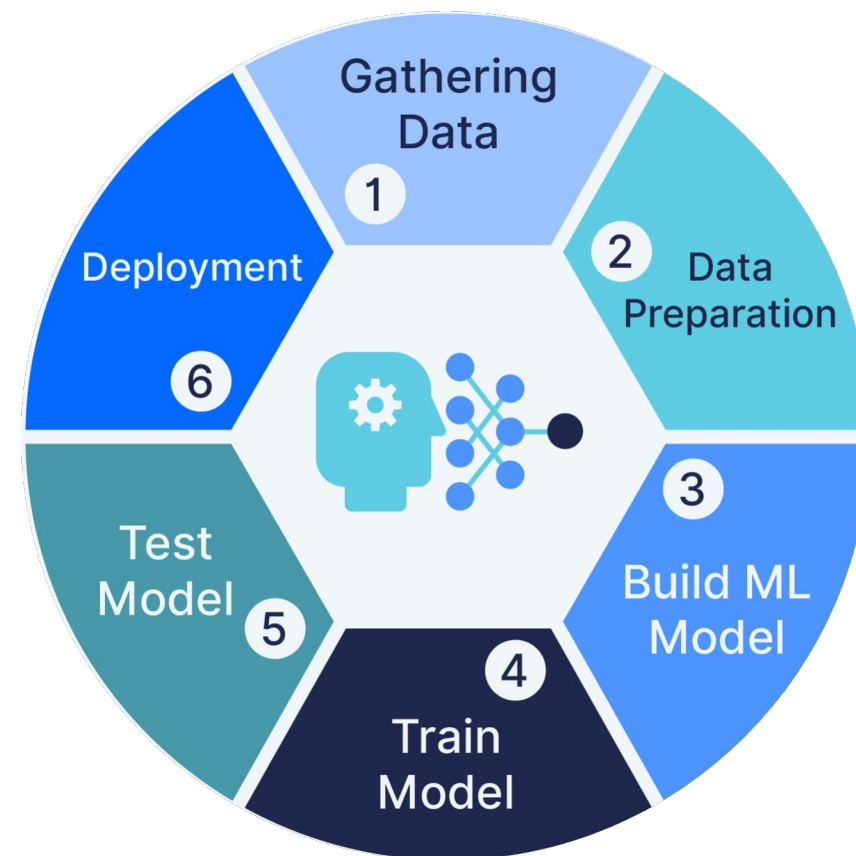
Инференс-серверы

Платформы инференса



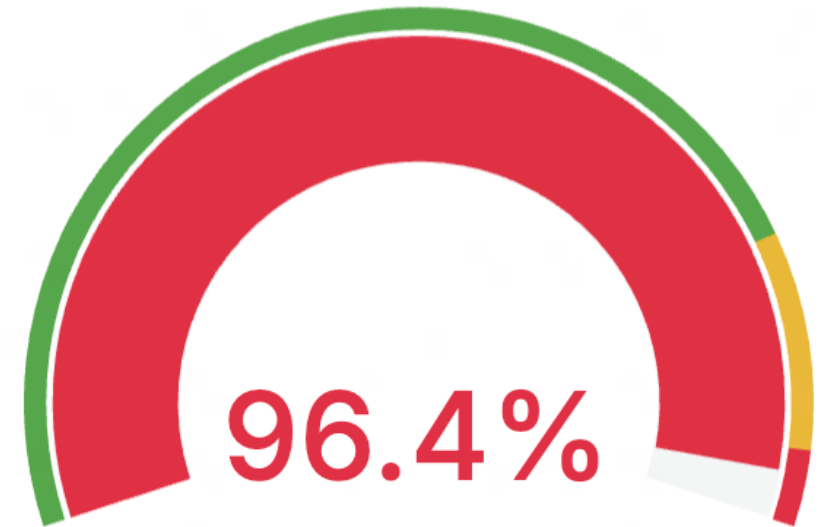
Автоматизация развёртывания

- Ручной и автоматизированный деплой;
- Поводы для регулярного обновления:
 - повышение качества модели;
 - data drift (пример с поиском);
- TTM – time to market:
 - сборка образа (кэширование);
 - выделение ресурсов;
 - скачивание и инициализация моделей;
- Быстрый деплой – доставка новых фичей, восстановление;
- Canary, Shadow deployment;



Утилизация GPU

- GPU – дорогой ресурс;
- Соблюсти SLA (время ответа, доступность)
+ минимизировать ресурсы;
- Выбор ресурса: CPU vs GPU, T4 / A40 / H100;
 - latency / throughput;
 - GPU память;
 - важность сервиса (кол-во инстансов);
- GPU Sharing: TimeSlicing, MPS, MIG;
- Учитывать паттерн инференса;



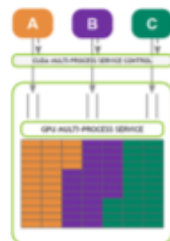
GPU Sharing

GPU “CONCURRENCY”

Choices



**Single Process
in CUDA**



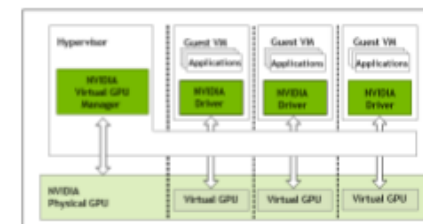
**Multi-Process
with CUDA MPS**



Time-slicing



MIG



Virtualization with vGPU

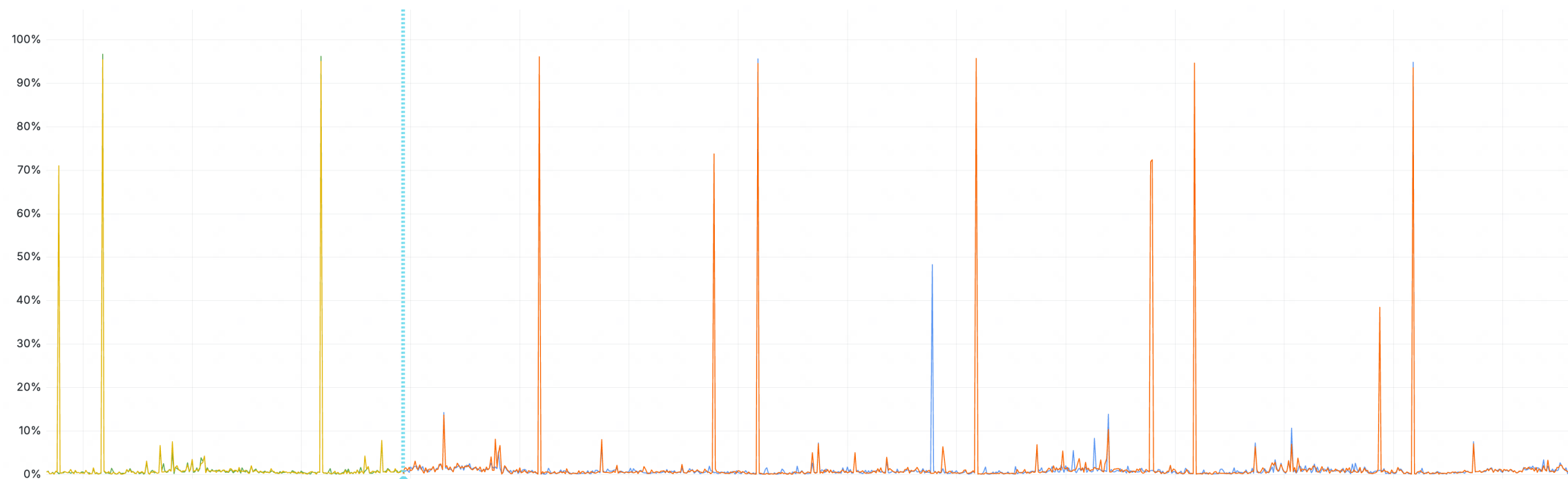
Application level

(using the CUDA programming
model APIs - CUDA streams)

GPU System Software / Hardware
(Mostly transparent to CUDA applications)

Пример несоответствия паттерну инференса

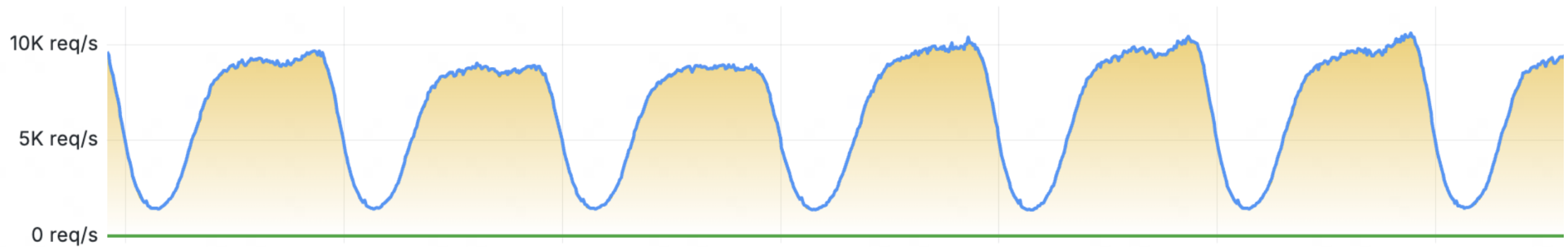
GPU Graphics Engine Utilization



Приоритезация сервисов

- Разные нагрузки: обучение, AI агенты, онлайн инференс;
- Важность в зависимости от требований к аптайму;
- Переменный трафик => масштабирование, выселение;

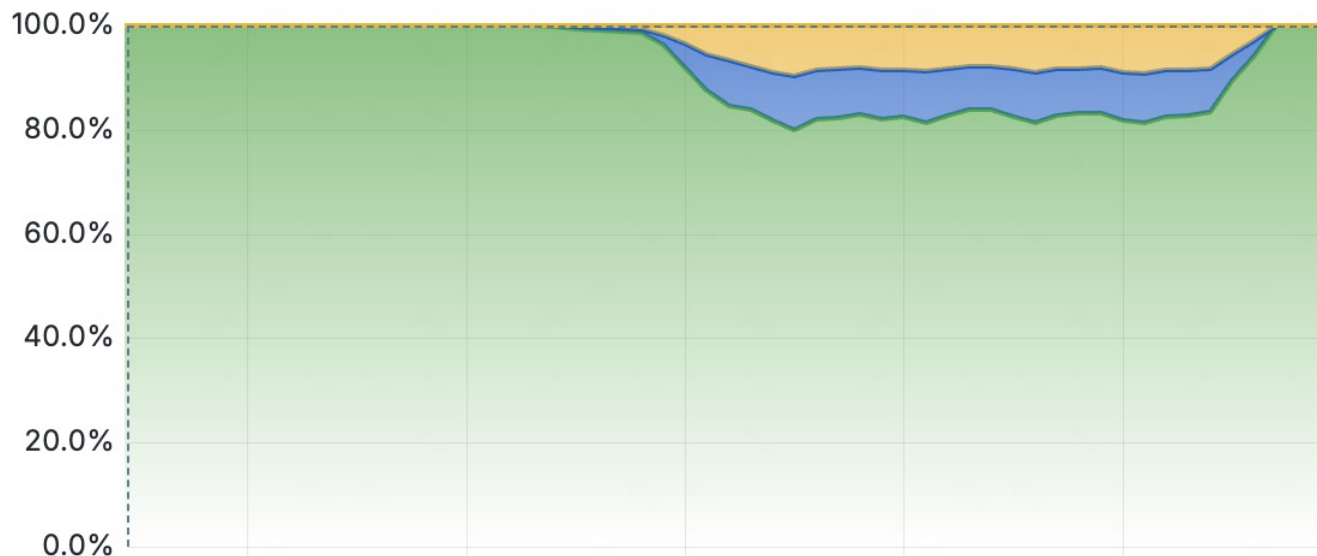
Incoming traffic [↗](#)



Мониторинг и стабильность

- Метрики сервиса:
 - время ответа;
 - доля ошибок;
 - утилизация ресурсов (лимиты);
- Алерты;
- Потеря одного ДЦ:
 - автомасштабирование;
 - возможна деградация, но не даунтайм;
 - восстановление;

Grpc response codes ratio [🔗](#)



Платформы инференса

- Управление инференс-сервисами;
- Абстракция от k8s кластера;
- Декларативное управление (YAML);
- Поддержка многих фреймворков;
- Multi-node inference;
- Стандартизация интерфейсов (Open Inference Protocol);
- Доставка моделей (S3, кэширование на нодах);
- Мониторинг;
- Балансировка и управление трафиком: Shadow, Canary;
- Автомасштабирование;
- KServe, Seldon Core, BentoML.

Выводы

- Паттерны инференса: синхронный, пакетный, потоковый;
- Фреймворки – PyTorch, ONNX, OpenVINO, TensorRT;
- Серверы – Triton Inference Server;
- Платформы – KServe, Seldon Core, BentoML;